

动态规划入门补充讲义

把暴力搜索树压缩成状态表

对 Day4 课件的二周目台阶补充

这节课先不背公式

- DP 的第一目标不是写出 $f[i] = \dots$ ，而是说清楚 $f[i]$ 是什么问题。
- 如果状态含义没定，转移式就是空中楼阁。别急，先把地基打好。

核心主线：暴力搜索会重复，DP 把重复的小问题只算一次。

DP 常出现在哪里

- 题目在求方案数、最大值、最小值、可行性。
- 暴力枚举方案可行但会反复进入同一个剩余问题。
- 答案能按最后一步、最后一个元素、选或不选来分类。
- 数据范围暗示不能指数枚举，通常需要 $O(n^2)$ 、 $O(nm)$ 、 $O(n \log n)$ 等级。

搜索树到状态图

1

暴力枚举所有选择

2

发现相同小问题重复出现

3

给小问题命名为状态

4

按依赖顺序只算一次

DP 不是神秘递推，而是把重复计算压扁。

写 DP 前的五件事

- 状态含义: $f[i][j]$ 用一句中文解释。
- 有效起点: 哪些状态一开始就知道。
- 转移来源: 当前状态按什么分类。
- 循环顺序: 算当前状态时, 依赖项是否已经算完。
- 答案位置: 输出一个状态, 还是所有状态取最值。

记忆化搜索 vs 递推

记忆化搜索

- 从原问题往下递归
- 第一次算出状态后存起来
- 适合依赖关系不容易手排时

递推填表

- 按依赖顺序从小到大填
- 每个状态显式循环计算
- 适合顺序清楚的基础模型

两者本质相同：都在状态图上求值。

模型一：斐波那契

- 暴力递归里 $\text{fib}(n - 2)$ 会被 $\text{fib}(n)$ 和 $\text{fib}(n - 1)$ 同时需要。
- 每个 $\text{fib}(i)$ 其实只需要算一次。
- 状态： $f[i]$ 表示第 i 项的值。
- 转移： $f[i] = f[i - 1] + f[i - 2]$ 。

斐波那契伪代码

```
f[0] = 0
```

```
f[1] = 1
```

```
for i from 2 to n:
```

```
    f[i] = f[i - 1] + f[i - 2]
```

```
answer = f[n]
```

真正重要的是：算 $f[i]$ 前， $f[i - 1]$ 和 $f[i - 2]$ 已经存在。

模型二：网格路径数

- 状态： $f[i][j]$ 表示从起点走到 (i, j) 的方案数。
- 最后一步只有两种来源：从左边来，或从下边来。
- 两类路径不重叠、不遗漏，所以方案数相加。
- 起点 $f[1][1] = 1$ 。

最后一步分类

```
f[1][1] = 1
```

```
for i from 1 to n:
```

```
    for j from 1 to m:
```

```
        if i == 1 and j == 1:
```

```
            continue
```

```
        f[i][j] = 0
```

```
        if i > 1: f[i][j] += f[i - 1][j]
```

```
        if j > 1: f[i][j] += f[i][j - 1]
```

方案数、最优值、可行性

目标	分类后怎么合并	例子
方案数	把每一类数量相加	网格路径数
最大/最小值	每类取最优，再取总最优	最大金币路径
是否可行	只要有一类可行即可	能否凑出容量

模型三：最大金币路径

- 状态仍然是 $f[i][j]$ ，但含义变成走到 (i, j) 的最大金币数。
- 最后一步还是左边或下边。
- 这次不能相加，因为我们只要最好的那条路。
- 转移： $value[i][j] + \max(\text{两个来源})$ 。

模型四：LIS 为什么要以 i 结尾

不够用的状态

- $f[i]$ 表示前 i 个数的 LIS
- 只知道长度，不知道末尾
- 无法判断能不能接上 $a[i]$

补信息后的状态

- $f[i]$ 表示以 $a[i]$ 结尾的 LIS
- 末尾固定，合法性可判断
- 答案是 $\max(f[i])$

LIS 转移

```
for i from 1 to n:
    f[i] = 1
    for j from 1 to i - 1:
        if a[j] < a[i]:
            f[i] = max(f[i], f[j] + 1)

answer = max(f[1..n])
```

状态要记录足够信息，让下一步判断是否合法。

背包：不要记录选了哪些

- 暴力状态如果记录完整选择集合，就是 2^n 。
- DP 只记录：处理到第几个物品、还剩多少容量。
- 状态： $f[i][j]$ 表示只考虑前 i 个物品，容量不超过 j 的最大价值。
- 当前物品只有两类：选，或不选。

01 背包二维转移

```
f[i][j] = f[i - 1][j]

if j >= volume[i]:
    f[i][j] = max(
        f[i][j],
        f[i - 1][j - volume[i]] + value[i]
    )
```

第 i 个物品最多选一次，所以来源必须是 $i - 1$ 。

一维优化的方向

01 背包：倒序

- 物品只能用一次
- 不能让本轮更新后的状态再拿同一物品
- j 从 m 到 $\text{volume}[i]$

完全背包：正序

- 物品可以用多次
- 允许本轮新状态继续转移
- j 从 $\text{volume}[i]$ 到 m

不要死背方向，问：同一物品能不能在本轮继续使用？

01 背包一维伪代码

```
for each item i:
    for j from m down to volume[i]:
        f[j] = max(
            f[j],
            f[j - volume[i]] + value[i]
        )
```

完全背包一维伪代码

```
for each item i:  
    for j from volume[i] to m:  
        f[j] = max(  
            f[j],  
            f[j - volume[i]] + value[i]  
        )
```

常见翻车点

- 只写 $f[i][j]$ ，没有中文状态含义。
- 初始化随手写 0，导致不可达状态被当成合法状态。
- 忘记答案位置：LIS 不是天然输出 $f[n]$ 。
- 一维背包循环方向写反。
- 状态漏信息，导致转移时还要追问历史细节。

比赛识别信号

- 前 i 个、两个前缀、以 i 结尾、走到某个格子。
- 路径方向受限，或选择过程天然有顺序。
- 每个物品选或不选，容量或资源有限。
- 求方案数、最大值、最小值、是否可行。

先写四行：数据范围、暴力复杂度、状态含义、转移分类。

最后的话

- DP 的难点不是数组，而是把问题压缩成状态。
- 先从暴力搜索出发，再找重复子问题。
- 每个状态都要像一个封装好的问题：含义清楚，答案可复用。
- 公式只是最后的记录。前面的分析才是你比赛时能迁移的东西。